



Hierarchical Clustering-Based Graphs for Large Scale Approximate Nearest Neighbor Search

Javier Vargas Muñoz^{a,*}, Marcos A. Gonçalves^b, Zanoni Dias^a, Ricardo da S. Torres^a

^a University of Campinas, Av. Albert Einstein, 1251, Campinas, São Paulo, Brazil

^b Universidade Federal de Minas Gerais, Av. Antônio Carlos, 6627, Belo Horizonte, Minas Gerais, Brazil

ARTICLE INFO

Article history:

Received 28 August 2018

Revised 20 March 2019

Accepted 15 July 2019

Available online 15 July 2019

Keywords:

Approximate nearest neighbors search

Graph-based search

Hierarchical clustering

Guided search

ABSTRACT

This paper presents a novel approach to perform fast approximate nearest neighbors search in high dimensional data, using a nearest neighbor graph created over large collections. This graph is created based on the fusion of multiple hierarchical clustering results, where a minimum-spanning-tree structure is used to connect all elements in a cluster. We propose a novel search technique to guide the navigation on the graph without computing exhaustively the distances to all neighbors in each step of the search, just to those in the *direction* of the query. The objective is to determine the nearest point to the query with a few number of distance calculations. We experimented in three datasets of 1 million SIFT, GIST, and GloVe features. Results show better speedups than another graph-based technique, and competitive speedups at high recall values when compared to classic and recent state-of-the-art techniques.

© 2019 Elsevier Ltd. All rights reserved.

1. Introduction

Nearest neighbors search is a broadly used component in many Information Retrieval (IR), Computer Vision, and Machine Learning (ML) tasks. Therefore, its fast execution is of paramount importance, especially when we are working with large collections of high dimensional data. Data structures for exact nearest neighbors (NN) search [1] are very efficient when data dimensionality is low, but they suffer dramatically with the course of dimensionality when the data dimension increases. Approximate nearest neighbors search techniques are suitable when they yield acceptable results with a small loss of precision, in return of a fast search. In this paper, we focus on Approximate Nearest Neighbors Search (ANNS).

Techniques for ANNS can be grouped into four categories: tree indexing schemes, hash-based solutions, product-quantization-based, and neighborhood graphs. The first presents a widely-used way to organize data. At each tree level, data are split into subsets, based on some criteria, which are then recursively applied to each subset until some stop condition is reached. Search is performed traversing from the root to the leaves. Usually, tree indexing techniques present cheap costs for index construction. Within

this category, popular methods include: KD-Tree [1], Hierarchical K-Means [2], FLANN [3], and VPTree [4].

On the other hand, hashing-based techniques aim to find a hash function that maps similar objects to near positions in the hash table. Then, search can be performed around the query position in the hash table. The use of multiple hashing functions increases the probability of finding the nearest neighbor, but also increases the storage cost to save the hash tables. Locality Sensitive Hashing (LSH) [5] is one of the best-known methods of this group. Some variants of LSH were proposed with good results as well [6,7].

Similarly to hashing based, Product-Quantization (PQ)-based techniques aim to encode an object using a compact representation (at most hundred of bits), but differently from hashing, to order collection objects, distances to the query are calculated by means of a very fast approximate computation. The principal advantage of this family of methods relies on scalability aspects, given that is not necessary to load the feature vectors of all collection objects, just their encoded representation. Since the first work [8], several variants were presented in the literature [9–12], with good results. Another group of related techniques are based on the use of sparse coding representations [13,14].

Recently, the use of nearest-neighbors graphs has attracted a lot of attention. Results presented in recent works [15–17] show significant gains of graph-based approaches over hash-based and space-partitioning trees techniques, in terms of search efficiency. Greedy search on NN graphs is conducted by moving towards to the closest neighbor to the query in a sequence of steps. At each step, to determine which is the closest neighbor, most of existing

* Corresponding author.

E-mail addresses: javier.munoz@ic.unicamp.br (J. Vargas Muñoz), mgoncalv@dcc.ufmg.br (M.A. Gonçalves), zanoni@ic.unicamp.br (Z. Dias), rtorres@ic.unicamp.br (R. da S. Torres).

methods perform an exhaustive search in the neighborhood (brute force). This can slow down the process of convergence to high recall values when the graph is dense.

In this paper, we introduce Hierarchical-Clustering-based Nearest Neighbor Graph (HCNNG), a novel graph-based framework for implementing ANNS. The novelties of our solution are:

- The *graph construction algorithm*. We use clustering procedures to identify close objects in the feature space. We then explore the proximity relationship between elements in each cluster to construct a sparse and fast navigable graph of nearest neighbors.
- The *search algorithm*. We present a novel approach to perform a *guided navigation* on the graph of nearest neighbors, avoiding the exhaustive computation of all neighbors' distances at early steps of the search.

Experiments conducted on collections comprised of image and textual feature vectors show that the proposed method yields better or comparable performance than recent state-of-the-art techniques (including methods from all families), for different kinds of searches (e.g., 1-NN, 10-NN, 100-NN).

The rest of the paper is organized as follows: Section 2 presents related work. Section 3 describes the graph construction algorithm proposed and introduces the directed search algorithm. Section 4 details the experimental protocol and obtained results. Finally, Section 5 presents our conclusions and draws possible future work.

2. Related work

Related work for ANNS can be grouped into four families of approaches: tree partitioning, hash-based, product quantization, and NN-graph-based methods, which are described next.

2.1. Tree partitioning approaches

A strategy broadly studied in the literature for NN search is to organize data in a tree data structure. Data are partitioned into subsets, based on some criteria, at each tree level until some stop condition is satisfied, e.g., a minimum size is reached. When a search is performed, the tree is traversed from the root to the leaves, which probably contain the closest point. A backtracking approach can be used to further explore other nodes and increase the probability of finding the most similar objects.

Tree structures for exact search (e.g., KD-Tree [1], Ball Tree [18], and Cover Tree [19]) are very efficient for low dimensional data, but their performance decreases quickly when dimensionality increases. KD-Tree is one of the most cited tree structures for exact search. In the algorithm of KD-Tree index construction, at each level and in a sequential order, a dimension is selected to split the data and a point is used to better balance the division. A well-known variant of KD-Trees for ANNS is named Randomized KD-Trees [20]. In their construction, several KD-Trees are created, where the dimension to split the data is selected randomly. Then, at query time, search is performed on all the trees, and is stopped when a fixed number of leaves is explored. This method is implemented in the widely used Fast Library for Approximate Nearest Neighbors (FLANN) [3].

Muja and Lowe [2] proposed the hierarchical k-means tree, where, at each level, the data are split using the k-means algorithm into K subsets, and then the same algorithm is applied recursively to the subsets generated. The recursion is stopped when the size of the subset is less than K . In the traverse from the root, at each level, the branch with centroid closest to the query is taken. The exploration is stopped when a fixed number of nodes are visited.

Methods for tree partitioning can be roughly divided into two groups. Techniques that divide data points with respect to hyperplanes and clustering based. From the first group, we can mention KD-Trees [1,20], PCA-Tree [21], and RP-Tree [22]. In the second, we can find hierarchical clustering tree [3], hierarchical k-means tree [2], Ball Tree [18], Cover Tree [19], GNAT [23], MDF-tree [24], LB-tree [25], and VP Tree [4].

2.2. Hash-based approaches

The principle of hash-based techniques is to use a hashing function that generates near hash values for similar objects. Thus, low-dimensional mapped values can be used to perform efficient search. One of the seminal works was presented by Indyk and Motwani [26]. Currently, the most cited method in this family is probably Locality Sensitive Hashing [5] (LSH). In this technique, multiple hashing tables are created and used at the same time to obtain a reduced list of candidates for each function. Next, an exhaustive search is performed in all the list of candidates to find the nearest neighbors. Thus, the more hash tables are created, the higher the probability to determine the nearest points, but also, this increases linearly the memory consumption.

Lv et al. proposed Multi-Probe LSH [6], a method that reduces the high storage requirement by reducing the number of hashing tables. The authors' idea is based on the supposition that if a nearest neighbor is not in the same bucket as the query, then is highly probable that it is contained in close buckets. In this way, the algorithm makes a harder exploration of closest buckets, reducing the number of hash tables needed to achieve high recall values. Bawa et al. [7] presented a variant of LSH, which self tunes its parameters to the data. On the other hand, many supervised approaches have been proposed in the literature [27–31] which aim to learn hashing functions that encode original vectors in such a way that allow effective and efficient search in the coding space. For a detailed revision of the literature on supervised approaches, see the survey by Wang et al. [32].

2.3. Product-Quantization-based approaches

The main idea of this family of techniques consists in encoding original feature vectors as the combination of their quantizing representations using multiple codebooks, aiming to reduce the principal memory storage required for the feature vectors. The coding error is usually obtained through the Euclidean distance between the original vector and the encoded approximate representation. At search time, encoded representations are used to perform fast approximate distance calculations from the query to the collection objects, using lookup tables of precomputed values.

Jegou et al. introduced the first work, denominated *Product Quantization* (PQ) [8], where the dimensions of initial feature space are split orthogonally, and then, by applying the k-means, it is created a codebook for each partition. Ge et al. [9] presented an optimization, where prior to the feature space partition, is applied the same rotation to all vectors. Another variant was proposed by Kalantidis and Avrithis [10]. In their approach, instead of using a global rotation, different rotations are applied to near locally vector groups, obtaining better results than the previous ones, in terms of recall at search time and coding error. To avoid exploring all elements in the collection, they proposed the use of inverted indices [8,11] to explore only those objects near to the query. The low memory consumption of those methods has shown a great scalability (up to billions of feature vectors). On the other hand, worse effectiveness results in terms of precision were observed when compared with the proposed method and the best baselines considered in this work.

2.4. NN-graphs-based approaches

The main idea of NN-graph-based approaches is to create a graph, where each vertex represents a collection object, and edges are created between a vertex and its closest neighbors (under some similarity criterion). Then, a greedy search algorithm can be applied at query time, which starts in a random vertex and, at each step, moves to the closest neighbor to the query, until a convergence stopping criterion is reached (e.g., there is no neighbor vertex closer to the query than the actual vertex).

Harwood and Drummond [15] proposed an incremental algorithm to create an NN graph, called Fast Approximate Nearest Neighbour Graphs (FANNG). Initially, the set of vertices is composed of each object in the collection. Then, in each iteration, two vertices v_1 and v_2 are selected randomly, and a greedy search is performed using v_1 as starting vertex and v_2 as query. If the search fails to arrive at v_2 , an edge is added between the last node visited and v_2 . An important strategy employed refers to the deletion of *occluding* edges. This makes the graph to preserve the closest and the spreadest neighbors of each vertex, leading to a more efficient graph traversal. The search over the resulting graph is performed by means of a variation of greedy search, which is implemented by both applying backtracking to further explore the graph when a local optimum is reached, and limiting the number of distance calculations. This method achieves quickly high recall rates, but it is outperformed by the proposed approach at medium recall values. This kind of solution is important for the implementation of interactive image retrieval systems [33–35], a scenario for which it is acceptable to produce quite effective results in return of a fast search, since these results are iteratively improved with the user interaction.

Malkov et al. [36] introduced the Small World Graphs (SW-graph). Similar to the FANNG construction algorithm, initially it is created an empty graph (no vertices or edges). In each step, a new vertex (object collection) is selected and a search over the actual graph to find a fixed number of its nearest neighbors is performed. The new vertex is added to the graph and non-directed edges are created between this vertex and the set of nearest neighbors found. This is repeated until all collection objects are included in the graph. Their objective is to create an approximation to the Delaunay graph [37], and, at the same time, maintain “long” edges to allow logarithmic navigation on the graph. This property is known as *Small World* [38]. However, in the final graph, many vertices end with high degree, increasing the number of distance calculations to reach the nearest neighbors.

A recent proposed approach, Hierarchical Navigable Small World (HNSW) [16], creates a hierarchy of NN graphs, in which each collection object is assigned to a maximum hierarchy level. Long edges (edges that connect distant objects) are presented in top layers, and the short ones in bottom layers. The construction of graphs is analog to SW. Incrementally, objects are added into the graphs, starting in the graph at the object’s maximum level and descending up to the graph in the ground layer, linking them to the nearest ones in each level. At query time, search starts in some vertex in the top layer’s graph and traverse the graph to find the closest ones to the query. When a local optimum is reached, one level is descended in the hierarchy, and search is started using as starting points the nearest vertices found at the above level. This is repeated until the ground layer is reached. This method showed to be one of the most competitive baselines in our experiments across all datasets.

Unlike the incremental strategy used by the aforementioned initiatives, the algorithm for graph construction proposed by Dong et al. [17] (KGraph) initially assumes a random set of neighbors for every vertex. In each iteration, based on the heuristic – *a neighbor of a neighbor is also likely to be a neighbor* –, these sets are updated

by selecting the nearest points from the actual set and the neighbors of neighbors. The process is repeated until the sets of neighbors of each vertex do not change significantly. Although the construction algorithm converges fast and approximates with high recall the real nearest neighbors of each object collection, the resulting graph does not present good properties for search, as it aims to construct an exact NN graph and experimental results showed low performance for search on these graphs (see Section 4.6).

The idea of creating a NN graph using the results of multiple clustering executions was explored previously by Wang et al. [39]. Their proposed method aims to create efficiently a near-exact NN graph, maintaining a list of the K-NN for each point. These lists are updated in each clustering execution by creating a complete subgraph for each cluster and, then, maintaining for each vertex, those new neighbors that are closer than any of their previously found K-NN. The clustering procedure is stopped when there is not a significant modification of K-NN lists in the current iteration. Then, aiming to discover and include even more true NN on those lists, these are improved through a neighbor propagation phase. Similar to KGraph, this method also aims to create an exact NN graph, and, consequently, experimental results showed inferior search performance than our proposed method.

Most of the algorithms used to search on NN graphs combine variations of greedy search with backtracking with multiple random-starting search to increase the probability of finding the true nearest neighbors. Results presented in the described previous works using a recent benchmark¹ have shown a fast convergence of NN-graphs-based techniques to reach high recall values when compared with tree space partitioning techniques and hash-based methods.

Differently from techniques described above, the proposed graph construction uses a *divide-and-conquer* strategy to create sparse subgraphs in reduced sets of close points (clusters), and then join all subgraphs to create a single global sparse NN graph. At query time, rather than taking a random or a fixed vertex to start the search, we choose a vertex with good probability to be near to the query point, by using multiple global KD-Tree’s. Then, at early steps of search, neighbors of vertices are not fully explored by using a geometry property of Euclidean space. Instead, an efficient guided search is performed to traverse the search space. Next, we describe in details our proposal.

3. HCNG: Graph-based guided approximate NN searches

This section introduces the proposed framework for ANNS, denoted as HCNG, which is based on an NN graph construction, resulting after performing multiple hierarchical clustering procedures (Section 3.1), and on the use of a guided search in the created graph (Section 3.2).

3.1. Graph construction

The graph construction process relies on two steps: **the execution of multiple hierarchical clustering procedures** and **the fusion of graphs defined by the obtained clusters**. The objective is to generate a graph with a good connectivity among data points. We need enough connections among the vertices for supporting effective searches, but, at the same time, without unnecessary multiple paths, which may lead to inefficient processing time.

The hierarchical clustering performed over a set of points defines an implicit relationship of proximity between the points in each cluster. We explore this idea to create an NN graph.

¹ ANN Benchmark: <https://github.com/erikbern/ann-benchmarks> (As of July 2019).

Algorithm 1 describes the hierarchical clustering procedure. Given a set of points P , a distance function D , and a minimum size of clusters n , if the size of P is lower than n , all points in P are connected by a variant of the minimum spanning tree (MST3), according to which the maximum degree of each vertex is limited to 3 (line 5). Otherwise, two points are selected randomly (lines 8 and 9) and the hierarchical clustering procedure is applied recursively over the set of points nearer to both selected points (lines 10 and 11). The main reason for using MST-like structures relies on the fact that **MST3 guarantees the connectivity between every pair of points in a cluster** (there is always a path), **keeps vertices with low degrees**, and points will be connected with others close to them. The maximum degree of 3 was determined experimentally, we tested with values ranging from 2 to 10, and observed that recall values does not change significantly for values greater than 3, thus, to guarantee lower vertices degree we used 3 for all the experiments presented in Section 4.

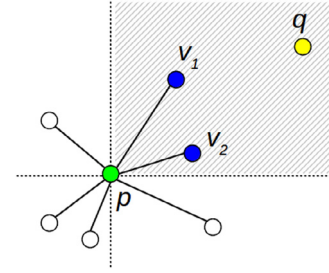


Fig. 1. Example of directed selection of neighbors.

tance function $D : \mathbb{R}^n \times \mathbb{R}^n \rightarrow \mathbb{R}^+$. The fusion graph G' is defined as $G' = (V', E')$, such that $V' = \cup_{i=1}^n V_i$ and $E' = \cup_{i=1}^n E_i$.

We choose this hierarchical-clustering-based approach instead of other methods like k-means, because it is hard to estimate the initial number of clusters and the number of iterations, without previous knowledge of the data distribution. It is easier to set the size of each cluster based on the number of neighbors we want to connect each vertex. The expected time complexity of each clustering execution scales in a logarithmic factor to the collection size leading to a total of $O(Nd \log(N/n))$, plus $O(Ndn \log(n))$ to construct MST3's for each cluster, where N is the size of the collection, n the estimated size of clusters, and d represents the cost of each distance calculation (equal to the data dimensionality for the Euclidean distance). Therefore, for m clustering executions the expected complexity of graph construction is $O(m \times (Nd \log(N/n) + Ndn \log(n)))$. With respect to the vertices degree, as we limited the vertex degree to 3 in each cluster, all vertices will have no more than $3 \times m$ neighbors in final graph, then the final number of edges complexity will be $O(Nm)$.

3.2. Guided search

The greedy approach to perform a search over an NN graph consist in, randomly, selecting a starting vertex and then, at each step, moving to the neighbor closer to the query. This is repeated until there is no neighbor closer to the query than the actual vertex. Some variants were proposed, as in FANNG [15], in which the authors used a priority queue to save the visited points and apply backtracking when a local minimum is reached. Also, in SW's [36], the search algorithm uses multiple random starting points. In both cases, neighbors of each vertex visited are fully explored, that is, distances are computed from all neighbors to the query. When the graph contains vertices with too many neighbors (hubs), this full exploration strategy will increase significantly the database scan rate to find the nearest neighbors.

The main idea of the proposed *guided search* is **to avoid exhaustively computing the distance from the query to all neighbors in each step of the search**. Our objective is to **focus the graph traversal process (search) on a reduced set of neighbors that are probably in the direction to the query**. Fig. 1 illustrates this concept. At any point in the navigation (say p), we will just compute the distances from the query (q) to those neighbors located **at the same quadrant (v_1 and v_2)** as the query (quadrant highlighted in the figure), in reference to the actual vertex.

Fig. 2 shows an example of multiple steps of a guided search from the starting point (green vertex) to the query point (yellow star), where the vertices explored by guided search are colored in blue and green (total 5), and vertices explored by classical greedy approach are those colored in gray in addition to the blue and green ones (total 10). The guided search starts at vertex 11 and, in the first step, it is computed the distance from the query to just vertices 6 and 8. Vertex 8 is then selected for the next step. In the second step, it is explored vertex 5, which is selected for the next

Algorithm 1: Hierarchical clustering procedure.

```

1 Function HierarchicalClustering( $P, n$ )
   Data: data points  $P$ , min size of clusters  $n$ 
   Result: graph edges  $E$ 
2    $N \leftarrow |P|$ 
3    $E \leftarrow \phi$ 
4   if  $N < n$  then
5      $E \leftarrow \text{MST3}(P)$ 
6   else
7     select randomly  $p_1$  and  $p_2$  points from  $P$ 
8      $P_1 \leftarrow \{p \in P \mid D(p, p_1) < D(p, p_2)\}$ 
9      $P_2 \leftarrow \{p \in P \mid D(p, p_1) \geq D(p, p_2)\}$ 
10     $E_1 \leftarrow \text{HierarchicalClustering}(P_1, n)$ 
11     $E_2 \leftarrow \text{HierarchicalClustering}(P_2, n)$ 
12     $E \leftarrow E_1 \cup E_2$ 
13 return  $E$ 
```

By applying the clustering procedure just once, the resulting graph will be highly disconnected. We address this problem by performing multiple random clustering procedures and combining their resulting graphs. This also solves the problem associated with points located on the border of two or more clusters, i.e., these points may belong to different clusters despite being close in the feature space. The process to create the final proximity graph is outlined in Algorithm 2.

Algorithm 2: Fusion of graphs.

```

1 Function CreateHCNNG( $P, m, n$ )
   Data: data points  $P$ , number of random clusterings  $m$ , min
         size of clusters  $n$ 
   Result: graph edges  $E$ 
2    $E \leftarrow \phi$ 
3   for  $i \leftarrow 1$  to  $m$  do
4      $E_i \leftarrow \text{HierarchicalClustering}(P, n)$ 
5      $E \leftarrow E \cup E_i$ 
6 return  $E$ 
```

More formally, the graph fusion procedure can be defined as follows. Let $\mathcal{C} = \{c_1, c_2, \dots, c_n\}$ be a set with n clusters defined after performing multiple hierarchical clustering procedures. For each cluster $c_i \in \mathcal{C}$, we can create a proximity graph $G_i = (V_i, E_i)$, where V_i is a set of vertex defined by the points belonging to c_i , and E_i is the set of edges generated by $\text{MST3}(V_i)$ for a given dis-

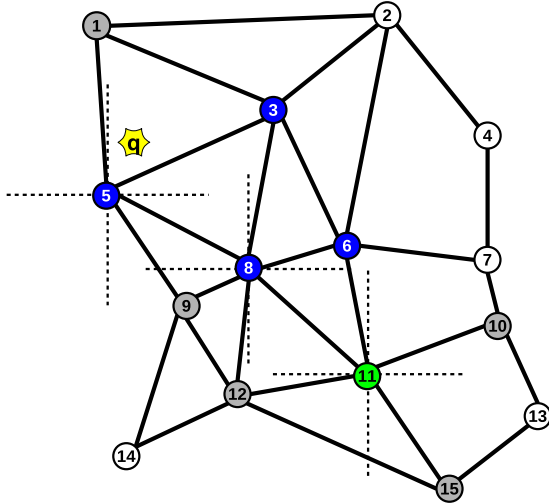


Fig. 2. Example of multiple steps in the guided search.

step, as it is closer to the query than vertex 8. Finally, vertex 3 is explored, and as it is not closer to the query than the actual vertex, the search ends. As we can observe, the number of distance computations is decreased in half compared to the classical greedy approach, in this example.

Determining the neighbors located in the same quadrant, at query time, is almost as expensive as computing all the distances, so we propose a preprocessing stage, to be applied on each vertex neighborhood N_p , with the objective of organizing the neighbors of each vertex p in quadrants (a subspace), taking as origin the vertex itself. Our idea is inspired in the space partition procedure used by KD-Trees. We also used a tree-like structure to store the neighbors in each subspace, where each internal node keeps the dimension used to partition the space, and two sub-trees to store the neighbors located in the positive and negative subspaces defined by the dimension chosen and an origin of reference (the vertex itself p). Leaf nodes contain the neighbors located in the subspace determined by the successive space partitions performed since the root up to the leaf. This process is outlined in Algorithm 3.

Algorithm 3: Creation of a tree structure to search in subspaces.

```

1 Function DivideSubspace( $p, N_p$ )
   Data: point of graph  $p$ , neighbors points  $N_p$ 
   Result: space partition tree  $T$ 
2    $dim \leftarrow$  select dimension based on  $N_p$ 
3    $neg \leftarrow \{v \in N_p | v[dim] < p[dim]\}$ 
4    $pos \leftarrow \{v \in N_p | v[dim] \geq p[dim]\}$ 
5   if  $|neg| = 0$  or  $|pos| = 0$  then
6      $T.points \leftarrow N_p$ 
7   else
8      $T.dim \leftarrow dim$ 
9      $T.neg \leftarrow DivideSubspace(p, neg)$ 
10     $T.pos \leftarrow DivideSubspace(p, pos)$ 
11  return  $T$ 

```

For each call to the *DivideSubspace* function, first a new dimension (dim) is selected to split the subspace. To maintain the tree balanced, we select the dimension that keeps the best balance between the number of neighbors in the negative and positive side (line 2). Then, we divide the neighbors into two subsets (neg and pos), using the dimension chosen. If one of these sets ends empty

after the subspace division (line 5), a leaf is created to store the actual set of neighbors (N_p) (line 6). Otherwise, an internal node is created to store the dimension used in the division, and then, recursively, two sub-trees ($T.neg$ and $T.pos$) are created (lines 9 and 10) to continue the division of the two subsets defined by neighbors neg and pos . A naïve algorithm for dimension selection costs $d \times |N_p|$, being d the data dimensionality and N_p the vertex degree. As the height of tree scales logarithmically to the vertex degree, and the vertex degree scales linearly to the number of clustering executions (m), then the complexity of Algorithm 3 is $O(dm \log(m))$ for each vertex of the graph.

As previously mentioned, the subspace division is applied to every vertex and its neighborhood in a preprocessing phase, along with the NN graph creation. At query time, it is enough to traverse from the root towards the leaf that contains the neighbors located at the same subspace as the query. Algorithm 4 details this traversal procedure. We start at the tree root. If, at some time, we find a leaf, we return the neighbors contained in this subspace (line 4). Otherwise, we check whether the query point belongs to the negative (line 7) or positive (line 9) subspace defined by the dimension stored, and then continue the traversal process following in the direction of the adequate subspace.

Algorithm 4: Identification of neighbors at the same subspace of the query.

```

1 Function GetNeighbors( $q, p, node$ )
   Data: query point  $q$ , point of graph  $p$ , node of space partition tree  $node$ 
   Result: neighbors in query's subspace  $N_q$ 
2    $N_q \leftarrow \emptyset$ 
3   if  $node$  is a tree leaf then
4      $N_q \leftarrow node.points$ 
5   else
6     if  $q[node.dim] < p[node.dim]$  then
7        $N_q \leftarrow GetNeighbors(q, p, node.neg)$ 
8     else
9        $N_q \leftarrow GetNeighbors(q, p, node.pos)$ 
10  return  $N_q$ 

```

The creation of subspace trees is performed in such a way that the resulting tree is balanced and, therefore, the expected height of trees is proportional to the logarithm of the number of neighbors (for a certain vertex). Note, also, that determining the points at the same query's subspace takes as many comparisons (of integers) as the tree height, and, the resulting graph of nearest neighbors remains sparse $O(m)$, as discussed in the previous section. Therefore, the application of the function *GetNeighbors* at each step of the navigation on the graph has a negligible cost.

A good non-randomized starting vertex selection for search could help to reduce the number distance calculations. We experimented creating a KD-Tree (offline) over all the collection and taking as starting point the leaf that contains the query. This idea was explored in a previous works [40]. Also, we do not actually perform the search on the KD-Tree, we just perform a sequence of comparison of values from the root to the leaf that contains the query, which is limited by the height of the KD-Tree (scales logarithmically with dataset size), without spending any distance calculation in determining the initial point. Similar to the idea of picking multiple random starting points, we decided to create multiple global KD-Tree's and pick the closest point to the query among the ones returned by the KD-Tree's. This value was set experimentally and is detailed in the next section. At this extra step, we just spend as many distance calculations as the number of KD-Tree's created

(to determine which one is closer). A similar idea was explored by Wang and Li [41], to select initial candidates based on multiple KD-Tree's for search on NN graphs, but differently from our approach, they actually perform the search on KD-Tree's (which involves extra costs), and also employed them again in the traversal graph process.

We used a modified version of backtracking search described previously, starting at the multiple KD-Tree based point selection, and, in each step, instead of exploring all neighbors, we continue the search through the neighbors returned by the *GetNeighbors* function. In any step of the search, when it had not found a closer point to the query in the selected subspace, adjacent subspaces are explored, i.e., closer leaves in the subspace partition tree are traversed. Subspaces are explored until a neighbor closer to the query than actual vertex is found.

4. Experiments

This section discusses the performed experiments aimed to validate the proposed method. All experiments were conducted using the Euclidean distance.

4.1. Datasets

We experimented with two bases from BIGANN datasets² for approximate nearest neighbor search. One collection is composed of 1 million SIFT feature vectors (128 dimensions) to index construction, and 10 thousand of queries to evaluate performance. The other collection contains 1 million GIST feature vectors (960 dimensions) and 1 thousand queries. These datasets were used in previous works [8,15,42] to evaluate approximate nearest neighbors search techniques.

We also tested with a collection of Global Vectors [43] of textual features (GloVe, 100 dimensions) extracted from 2 billion of tweets.³ In this dataset, to maintain a similar size than in previous datasets, we used a randomly selected subset of 1 million vectors to index construction and 10 thousand queries.

4.2. Parameters tuning

Two parameters are used in the HCNG construction algorithm: the number executions of hierarchical clustering procedures m , and the minimum size of clusters n . We performed an exhaustive parameter search in a synthetic dataset of 100 thousand uniformly sampled random vectors with a dimensionality of 20. Fig. 3 shows the heat map obtained by evaluating at Recall@10 search results on graphs created for different values assigned to parameters m and n . To evaluate the recall of each configuration, we fixed the maximum number of distance calculation allowed in search (called as T from now on) to 1000. We can observe in the figure that the convergence of the recall scores is reached for $m \geq 20$ and $n \geq 1000$. We set the number of clustering procedure executions to 20 for all experimental results presented in the following sections. Even though we can get same recall values with a larger m and lower n , it is less costly to choose a lower number of clustering executions. In general, most of our experiments across different datasets suggested an optimal value of $\sqrt{N}$ for the minimum size of clusters, being N the size of the collection, so we use this value in all our experiments.

At search time, to determine the starting vertex, we experimented creating different number of KDTree's, from 1 to 20. To evaluate the impact of this parameter, we performed the search

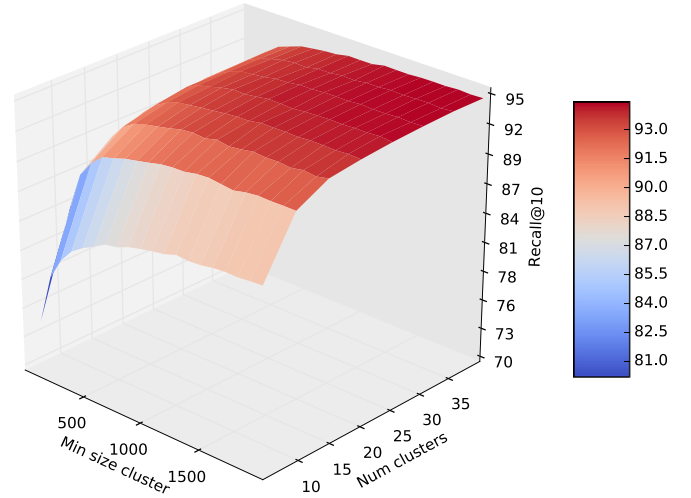


Fig. 3. Recall@10 for different values of clusters' minimum size and clustering executions.

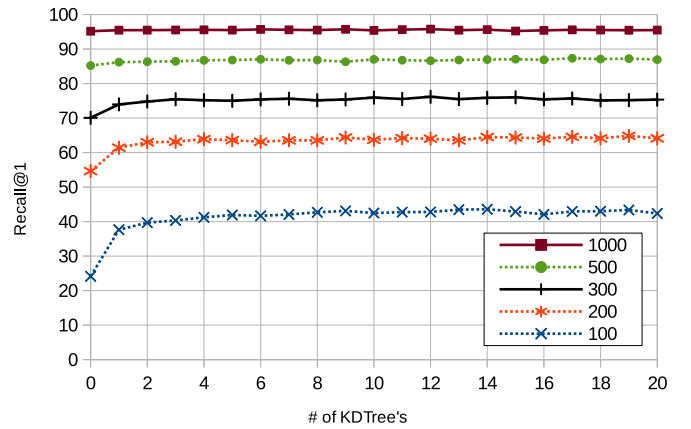


Fig. 4. Recall@1 for different numbers of KDTree's and maximum number of distance calculations (SIFT dataset).

(using the proposed guided strategy) on the graph created over SIFT dataset with the parameters suggested above, and measured the Recall@1. Also, we ran different experiments limiting to {100, 200, 300, 500, 1000} the value of T . Results for this experiments are shown in Fig. 4. As it can be observed, for lower values of T (greater speedups), there is a significant gain in recall when the number of KDTree's increases. Based on these results, we selected the value of 10 (number of KDTree's) for the next experiments, as we wanted that our method yields results with high values of recall for any desired speedup.

Fig. 5 shows results of proposed framework on SIFT dataset using different configurations for search. The same behavior was observed for the other datasets. It can be observed that the two proposed approaches (initial vertex selection based on multiple KDTree's, and guided search) obtain significant gains over a classical approach for search (random initial vertex selection with backtracking), and when combined (KDTree + Guided), improve even more the results.

4.3. Scaling datasets

We conducted an experiment to measure the average scan rate performed (proportion of the elements explored in some dataset) by 10-NN search (find 10 nearest neighbors) to achieve a 99% of recall in all datasets. We measure this for different dataset sizes

² BIGANN: <http://corpus-texmex.irisa.fr/> (As of July 2019).

³ GloVe: <https://nlp.stanford.edu/projects/glove/> (As of July 2019).

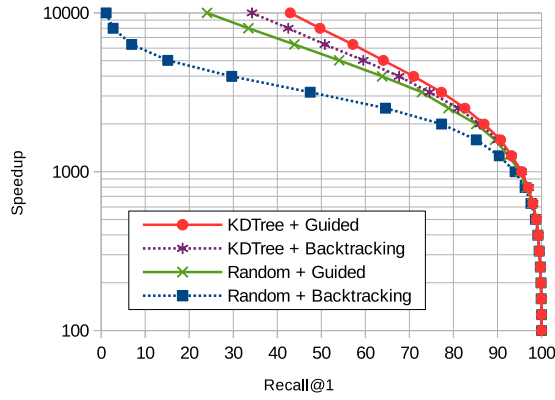


Fig. 5. Different configurations of search (starting vertex + exploration strategy), on SIFT dataset.

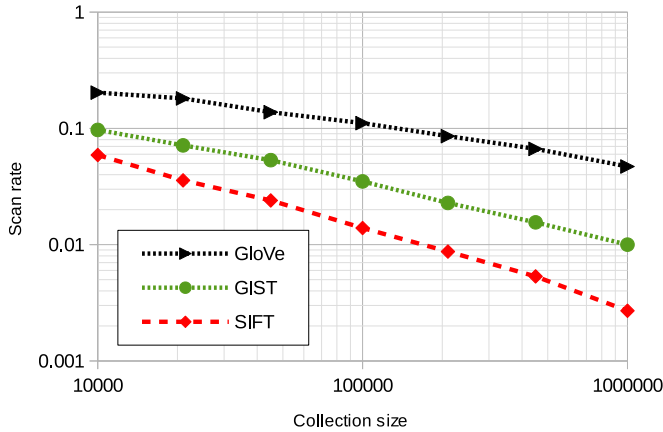


Fig. 6. Scan rate needed to reach 99% of recall for different collection sizes.

Table 1
Intrinsic dimensionality of datasets, estimated through Hausdorff dimension.

Base	GloVe	GIST	SIFT
Dim	15	9	10

and results are presented in Fig. 6, with both axes in logarithmic scale. The shapes of the curves suggest that the search of proposed method scales polylogarithmically to the dataset size, independently of the dataset. The proposed search method shows to be very efficient in SIFT dataset by just needing to scan less than 0.4% of all dataset at size 1 million. The slope of GloVe's curve shows a slower decreasing of scan rate than in the other collections. This seems contradictory to the fact that dimensionality in GloVe vectors is far smaller than GIST vectors and slightly different from SIFT vectors. However an analysis of the intrinsic dimensionality of the datasets showed that GloVe has the highest value of all 3 datasets. To estimate the intrinsic dimensionality we used a method based on the analysis of fractal dimensions, known as *Hausdorff dimension* [44]. These values are shown in Table 1.

4.4. Baselines

We compared the proposed approach, henceforth called HC-NNG, with several well-known and recent state-of-the-art methods. FLANN library [3] is probably the most known library for approximate nearest neighbors search. There are three main space partitioning trees techniques implemented in FLANN: Randomized KD-Trees [20], K-Means Tree [2], and Hierarchical Clustering Tree [45]. We used the auto-tuned algorithm from FLANN library

in our experiments, which selects the best algorithm (included in FLANN) and parameter values for each of the data.

Among the NN-graph-based techniques, we included as baseline a recently proposed method [15], called FANNG. We used our own implementation, and run the experiments with parameters reported by authors on the same datasets. Other baselines included from this family of methods were SW-graph [36] and HNSW [16]. For both, the experiments were conducted with the implementation found in a recent library called *Non-Metric Space Library* (NMSLIB).⁴ Also, we run the KGraph [17] technique, using the implementation provided by authors in their website.⁵ Finally, we implemented the method introduced by Wang et al. [39] and included it as baseline (denoted as Wang2012).

To compare with methods based on Product Quantization, we use a implementation available of the classic method and two more recent optimized versions⁶: Product Quantization (PQ) [8], Optimized Product Quantization (OPQ) [9], and Additive Quantization (AQ) [12].

4.5. Evaluation criteria

A widely-used evaluation measure used for approximate nearest neighbors search are Speedup $\times$ Recall charts. To keep the speedup independent of architecture where experiments were executed, we only consider the number of distance calculations performed by each method. Thus, speedup is defined by:

$$\text{Speedup} = \frac{\text{Collection size}}{\text{Number of distance calculations}}$$

Recall is defined by the fraction of true nearest neighbors that are successfully retrieved. We conducted three experiments to evaluate the methods' performance: the first to search for the closest neighbor (1-NN), the second to search the 10 nearest neighbors (10-NN), and the last to search the 100 nearest neighbors (100-NN).

4.6. Experimental results

Search performance results across SIFT, GIST, and GloVe are shown in Fig. 7. Results for 1-NN, 10-NN, and 100-NN searches are presented in the first, second, and third rows, respectively.

In the case of SW-graph technique, the graph is constructed with the objective of allowing a logarithmic navigation, but this property is negatively affected by the high degree of many vertices, leading to a slow convergence, which is reflected in the results of all experiments. As KGraph and Wang2012 aims to approximate an exact KNN graph, thus its performance should be similar to an exact KNN graph. In all datasets, we added an extra baseline, with the name of *kNN-Graph*, that represents the search performed over an exact KNN graph. As it can be observed, empirical results suggest a considerable margin of difference of kNN-Graph against methods that optimize graphs for NN search (e.g., HCNNNG, FANNG, and HNSW). Consequently, KGraph and Wang2012 have a similar behavior. Is worth it to stress out that, although, Wang2012 [39] employs a similar strategy to construct the NN graph, the selection of MST3 for connecting the points inside clusters, in our proposed approach, has a great impact on search results, demonstrating a good balancing between graph sparsity and an easy to traverse NN graph.

On the SIFT collection, although FLANN starts with one of the highest recall scores (at Speedup of 10^4), it evolves slowly, being quickly outperformed by the proposed method, HCNNNG, and

⁴ NMSLIB: <https://github.com/searchivarius/nmslib> (As of July 2019).

⁵ KGraph: <http://www.kgraph.org> (As of July 2019).

⁶ Quantizations: <https://github.com/arbabenko/Quantizations> (As of July 2019).

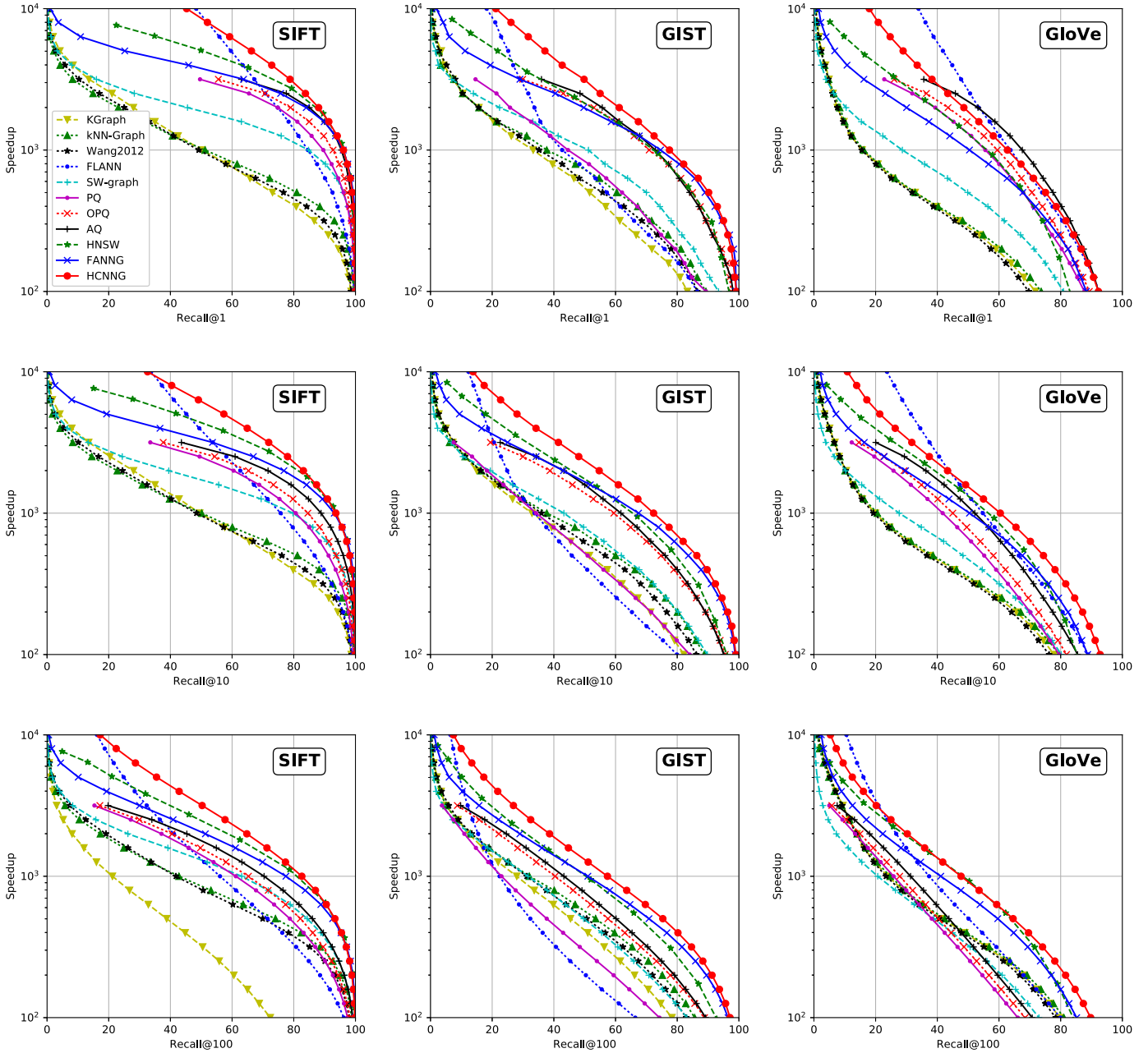


Fig. 7. Speedup vs recall on SIFT, GIST, and GloVe datasets.

most of baselines, for all the three cut-off points considered (1, 10, and 100). In this dataset, HNSW shows to be the most competitive baseline, and FANNG, the second best. Even though FANNG starts with low recall values, it quickly evolves to high scores ($> 90\%$). From the quantization-based techniques, the best method, AQ, shows to be competitive at high recall values at 1-NN search, but it is outperformed by a significant margin by HCNNG for 10-NN and 100-NN searches. In this dataset, HCNNG yielded best results both in terms of effectiveness and efficiency: it achieves high recall values quickly and yields good recall scores when the maximum number of distance calculation is low (higher speedups).

On the GIST dataset, the performance of all methods is clearly affected by its higher data dimensionality, but, similarly to the SIFT dataset, the relative performance of all methods does not change. Again, HNSW shows to be competitive, at the three cut-off points, but for high recall values when compared with FANNG. For high

recall values, FANNG presents similar results as the proposed HCNNG. AQ method shows good performance at first (1-NN search), but it is notably more affected than HCNNG when K (number of NN to search) increases. As in case of SIFT dataset, the curve of the proposed HCNNG was superior at almost any point for 1-NN, 10-NN, and 100-NN searches on the GIST dataset.

Results obtained by all evaluated methods on GloVe (for all 1-NN, 10-NN, and 100-NN searches) were significantly worse than in SIFT and GIST datasets, considering that they have the same size and GloVe has a lower dimensionality than SIFT and GIST vectors. This fact is explained by the intrinsic dimensionality of datasets. As seen in Section 4.3, GloVe has the highest intrinsic dimensionality, therefore evolution to get high recall values will be in fact slower. Although, FLANN and AQ yielded better results than HCNNG at medium recall values for 1-NN search, their performance is affected dramatically when the number of NN to search increases,

as it can be observed in 10-NN and 100-NN search results for this dataset.

Summarizing, HNSW and FANNG have shown to be the most competitive methods in all datasets, obtaining comparable speedups to the proposed HCNNG at high recall values, but they are outperformed at medium recall values by HCNNG. The FLANN's auto-tuned algorithm starts with good recall values, but, in most experiments, speedup tends to decrease quickly as the recall increases. In the case of AQ, although it has some competitive results at 1-NN search, the method performance is very sensitive to the number of NN to search. The proposed method shows the best behavior across all datasets, obtaining both good speedups at medium recall and best speedups at high recall values in the three types of searches performed, demonstrating its applicability in cases where not only the closest neighbor is required (e.g., local image features matching) but also when a set of them are requested (e.g., content-based image retrieval).

5. Conclusions

This paper presented a novel approach to create a nearest neighbor graph with good navigable properties, over large collections of high dimensional data. Our idea explores the relationship of proximity between points in the same cluster, after a set of clustering procedures. Another important contribution is the guided search algorithm that avoids exhaustive distance computations based on geometric properties of neighbors in the feature space. Experimental results show higher recall values than other graph-based techniques for high speedups (most notoriously in 1-NN search); and a comparable convergence to high recalls with better results than those observed for state-of-the-art techniques at searching different number of nearest neighbors.

As future work, we intend to evaluate HCNNG in other metric spaces, and investigate the possibility of a generalized version of our approach for non-metric spaces. Also, we plan to develop strategies for automatic selection of parameters in graph construction. In the search algorithm, we plan to explore other strategies (e.g., learning techniques) to guide the navigation on graphs, with the objective of accelerating even more the convergence to high recall values.

Declaration of Competing Interest

None.

Acknowledgements

Authors are grateful to CNPq (grants #307560/2016-3, #400487/2016-0, #425340/2016-3, #304380/2018-0, and #422593/2018-4), CAPES (grant #88881.145912/2017-01), FAPEMIG (grant PPM-00543-17), FAPESP (grants #2014/12236-1, #2015/24494-8, #2016/50250-1, #2017/20945-0, #2015/11937-9, #2017/12646-3, and #2017/16246-0) and the FAPESP-Microsoft Virtual Institute (grants #2013/50155-0, #2013/50169-1, and #2014/50715-9). This study was financed in part by the Coordenação de Aperfeiçoamento de Pessoal de Nível Superior - Brasil (CAPES) - Finance Code 001.

References

- [1] J.L. Bentley, Multidimensional binary search trees used for associative searching, *Commun. ACM* 18 (9) (1975) 509–517.
- [2] M. Muja, D.G. Lowe, Fast approximate nearest neighbors with automatic algorithm configuration, in: *Proceedings of the International Conference on Computer Vision Theory and Application*, 2009, pp. 331–340.
- [3] M. Muja, D.G. Lowe, Scalable nearest neighbor algorithms for high dimensional data, *IEEE Trans. Pattern Anal. Mach. Intell.* 36 (11) (2014) 2227–2240.
- [4] P.N. Yianilos, Data structures and algorithms for nearest neighbor search in general metric spaces, in: *Proceedings of the 4th Annual ACM-SIAM Symposium on Discrete Algorithms*, 1993, pp. 311–321.
- [5] A. Andoni, P. Indyk, Near-optimal hashing algorithms for approximate nearest neighbor in high dimensions, *Commun. ACM* 51 (1) (2008) 117–122.
- [6] Q. Lv, W. Josephson, Z. Wang, M. Charikar, K. Li, Multi-probe LSH: efficient indexing for high-dimensional similarity search, in: *Proceedings of the 33rd International Conference on Very Large Data Bases*, in: VLDB'2007, 2007, pp. 950–961.
- [7] M. Bawa, T. Condie, P. Ganesan, LSH forest: self-tuning indexes for similarity search, in: *Proceedings of the 14th International Conference on World Wide Web*, in: WWW'2005, 2005, pp. 651–660.
- [8] H. Jegou, M. Douze, C. Schmid, Product quantization for nearest neighbor search, *IEEE Trans. Pattern Anal. Mach. Intell.* 33 (1) (2011) 117–128.
- [9] T. Ge, K. He, Q. Ke, J. Sun, Optimized product quantization for approximate nearest neighbor search, in: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2013, pp. 2946–2953.
- [10] Y. Kalantidis, Y. Avrithis, Locally optimized product quantization for approximate nearest neighbor search, in: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2014, pp. 2329–2336.
- [11] A. Babenko, V. Lempitsky, The inverted multi-index, *IEEE Trans. Pattern Anal. Mach. Intell.* 37 (6) (2015) 1247–1260.
- [12] A. Babenko, V. Lempitsky, Additive quantization for extreme vector compression, in: *Proceedings of the Conference on Computer Vision and Pattern Recognition*, 2014, pp. 931–938.
- [13] A. Cherian, Nearest neighbors using compact sparse codes, in: *International Conference on Machine Learning*, in: ICML'2014, 2014, pp. 1053–1061.
- [14] Y. Zhang, M. Xiang, B. Yang, Graph regularized nonnegative sparse coding using incoherent dictionary for approximate nearest neighbor search, *Pattern Recognit.* 70 (2017) 75–88.
- [15] B. Harwood, T. Drummond, FANNG: fast approximate nearest neighbour graphs, in: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2016, pp. 5713–5722.
- [16] Y.A. Malkov, D. Yashunin, Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs., 2016 arXiv:1603.09320.
- [17] W. Dong, C. Moses, K. Li, Efficient k-nearest neighbor graph construction for generic similarity measures, in: *Proceedings of the 20th International Conference on World Wide Web*, in: WWW'2011, 2011, pp. 577–586.
- [18] S.M. Omohundro, Five balltree construction algorithms, *Int. Comput. Sci. Inst. Berkeley*, 1989.
- [19] A. Beygelzimer, S. Kakade, J. Langford, Cover trees for nearest neighbor, in: *Proceedings of the 23rd International Conference on Machine Learning*, in: ICML'2006, 2006, pp. 97–104.
- [20] C. Silpa-Anan, R. Hartley, Optimised KD-Trees for fast image descriptor matching, in: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2008, pp. 1–8.
- [21] R.F. Sproull, Refinements to nearest-neighbor searching in k-dimensional trees, *Algorithmica* 6 (1–6) (1991) 579–589.
- [22] S. Dasgupta, Y. Freund, Random projection trees and low dimensional manifolds, in: *Proceedings of the 40th Annual ACM Symposium on Theory of Computing*, in: STOC'2008, 2008, pp. 537–546.
- [23] S. Brin, Near neighbor search in large metric spaces, in: *Proceedings of the 21th International Conference on Very Large Data Bases*, in: VLDB'1995, 1995, pp. 574–584.
- [24] E. Gómez-Ballester, L. Micó, J. Oncina, Some approaches to improve tree-based nearest neighbour search algorithms, *Pattern Recognit.* 39 (2) (2006) 171–179. Part Special Issue: Complexity Reduction.
- [25] Y.-S. Chen, Y.-P. Hung, T.-F. Yen, C.-S. Fuh, Fast and versatile algorithm for nearest neighbor search based on a lower bound tree, *Pattern Recognit.* 40 (2) (2007) 360–375.
- [26] P. Indyk, R. Motwani, Approximate nearest neighbors: towards removing the curse of dimensionality, in: *Proceedings of the 30th Annual ACM Symposium on Theory of Computing*, 1998, pp. 604–613.
- [27] J. Wang, J. Wang, N. Yu, S. Li, Order preserving hashing for approximate nearest neighbor search, in: *Proceedings of the 21st ACM International Conference on Multimedia*, in: MM'2013, ACM, New York, NY, USA, 2013, pp. 133–142.
- [28] W. Liu, C. Mu, S. Kumar, S.-F. Chang, Discrete graph hashing, in: Z. Ghahramani, M. Welling, C. Cortes, N.D. Lawrence, K.Q. Weinberger (Eds.), *Advances in Neural Information Processing Systems*, Curran Associates, Inc., 2014, pp. 3419–3427.
- [29] T.-T. Do, A.-D. Doan, D.-T. Nguyen, N.-M. Cheung, Binary hashing with semidefinite relaxation and augmented lagrangian, in: B. Leibe, J. Matas, N. Sebe, M. Welling (Eds.), *European Conference on Computer Vision*, Springer International Publishing, Cham, 2016, pp. 802–817.
- [30] F. Shen, X. Zhou, Y. Yang, J. Song, H.T. Shen, D. Tao, A fast optimization method for general binary code learning, *IEEE Trans. Image Process.* 25 (12) (2016) 5610–5621.
- [31] V.E. Liong, P.M., Deep hashing for compact binary codes learning, in: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2015, pp. 2475–2483.
- [32] J. Wang, T. Zhang, J. Song, N. Sebe, H.T. Shen, A survey on learning to hash, *IEEE Trans. Pattern Anal. Mach. Intell.* 40 (4) (2018) 769–790.
- [33] D.C.G. Pedronette, R.T. Calumby, R. da S. Torres, A semi-supervised learning algorithm for relevance feedback and collaborative image retrieval, *EURASIP J. Image Video Process.* 2015 (1) (2015) 27.

- [34] M. Ferecatu, D. Geman, A statistical framework for image category search from a mental picture, *IEEE Trans. Pattern Anal. Mach.Intell.* 31 (6) (2009) 1087–1101.
 - [35] R.T. Calumby, R. da Silva Torres, M.A. Gonçalves, Multimodal retrieval with relevance feedback based on genetic programming, *Multimedia Tool Appl.* 69 (3) (2014) 991–1019.
 - [36] Y. Malkov, A. Ponomarenko, A. Logvinov, V. Krylov, Approximate nearest neighbor algorithm based on navigable small world graphs, *Inf. Syst.* 45 (2014) 61–68.
 - [37] F. Aurenhammer, Voronoi diagrams a survey of a fundamental geometric data structure, *ACM Comput. Surv.* 23 (3) (1991) 345–405.
 - [38] J. Kleinberg, The small-world phenomenon: an algorithmic perspective, in: *Proceedings of the 32nd Annual ACM Symposium on Theory of Computing*, 2000, pp. 163–170.
 - [39] J. Wang, J. Wang, G. Zeng, Z. Tu, R. Gan, S. Li, Scalable K-NN graph construction for visual descriptors, in: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, in: *CVPR'2012*, 2012, pp. 1106–1113.
 - [40] S. Arya, D.M. Mount, Approximate nearest neighbor queries in fixed dimensions, in: *Proceedings of the 4th Annual ACM-SIAM Symposium on Discrete Algorithms*, 1993, pp. 271–280.
 - [41] J. Wang, S. Li, Query-driven iterated neighborhood graph search for large scale indexing, in: *Proceedings of the 20th ACM International Conference on Multimedia*, in: *MM'2012*, ACM, New York, NY, USA, 2012, pp. 179–188.
 - [42] L. Paulevé, H. Jégou, L. Amsaleg, Locality sensitive hashing: a comparison of hash function types and querying mechanisms, *Pattern Recognit. Lett.* 31 (11) (2010) 1348–1358.
 - [43] J. Pennington, R. Socher, C. Manning, Glove: global vectors for word representation, in: *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing*, 2014, pp. 1532–1543.
 - [44] F. Hausdorff, Dimension und äußeres maß, *Mathematische Annalen* 79 (1) (1918) 157–179.
 - [45] M. Muja, D.G. Lowe, Fast matching of binary features, in: *Proceedings of the 9th Conference on Computer and Robot Vision*, 2012, pp. 404–410.
- JAVIER ALVARO VARGAS MUÑOZ** received his Bachelor degree in Informatic Engineering from Universidad San Antonio Abad del Cusco in 2013. He got a Master degree from University of Campinas in 2016. Actually, he is a Ph.D. candidate at University of Campinas. His research interests include multimedia retrieval and large-scale multimedia indexing.
- MARCOS ANDRÉ GONÇALVES** holds a degree in Computer Science from the Federal University of Ceara (1995), a Master's Degree in Computer Science from the University of Campinas (1997), and a Ph.D. in Computer Science from the Virginia Polytechnic Institute and State University (Virginia Tech). He is currently an Associate Professor at the Federal University of Minas Gerais. He has received several awards and honors throughout his career. He works in the area of Computer Science with emphasis on Information Retrieval and Machine Learning.
- ZANONI DIAS** received his B.Sc. in Computer Science (1997) from the University of Campinas and his Ph.D. in Computer Science from the same university (2002). Currently, he is Associate Professor at the Institute of Computing, University of Campinas, Brazil. His research interests include Computational Biology, Data Science, and Machine Learning.
- RICARDO DA SILVA TORRES** is a Full Professor in the Department of Information Systems at the Institute of Computing, University of Campinas, and affiliated with the Reasoning for Complex Data (RECOD) Lab. He received a B.Sc. in Computer Engineering from the University of Campinas, Brazil, in 2000 and his Ph.D. degree in Computer Science at the same university in 2004. His research interests include Data Science, Multimedia Data Analysis, Multimedia Retrieval and Classification, Machine Learning, Visual Computing, and eScience.